Table 5.3 Sample authorization decisions using RBAC. This table presents scenarios related to the RBAC permissions shown in figure 5.2. Each decision is based on the roles assigned to a principal and the permissions granted through those roles or their inherited roles.

Scenario	Principal	Requested Action	Decision	Reason
Can Bob approve a code check-in?	Bob	`approve-code-check-in`	Allowed	Bob has the `engineer` role, which includes this permission.
Can Alice approve a code check-in?	Alice	`approve-code-check-in`	Allowed	`senior-engineer` inherits from `engineer`, so Alice inherits the permission.
Can Alice deploy to production?	Alice	`deploy-to-production`	Allowed	`senior-engineer` grants this permission directly.
Can Bob deploy to production?	Bob	`deploy-to-production`	Denied	`engineer` does not grant this permission.
Can Carol view tickets?	Carol	`view-tickets`	Allowed	`support-lead` inherits from `customer-support`, which grants this permission.
Can Carol delete tickets?	Carol	`delete-tickets`	Allowed	`support-lead` grants this permission directly.
Can Bob delete tickets?	Bob	`delete-tickets`	Denied	engineer has no ticketing roles; no permission granted.

As the scenarios in table 5.3 illustrate, RBAC enables consistent and predictable authorization decisions by abstracting permissions through roles. Instead of scattering authorization logic across individual services or hardcoding rules in applications, organizations can depend on a centralized role structure that reflects how work is organized. While RBAC doesn’t support dynamic conditions like time of day or device posture, its simplicity and alignment with organizational structure make it a dependable choice in many environments, especially when paired with well-governed role definitions and assignment processes.